CHAPTER 3 A Ray-Tracing-Enhanced Lagrangian Particle Tracking Framework for Ice Accretion Applications

This chapter presents the development of a Lagrangian particle-tracking solver for aircraft ice accretion simulations on distributed-memory systems. The primary aim of this work is to enhance the robustness, accuracy, and computational efficiency of Lagrangian droplet tracking. To this end, a face-to-face tracking scheme is employed for robust boundary-intersection detection, combined with a ray-tracing-enhanced bilinear-patch intersection technique to improve computational performance and scalability. Improvements to the droplet-seeding strategy are also introduced to facilitate the initialization and refinement of the seeding grid. Supercooled large droplet (SLD) effects are incorporated to improve agreement with experimental collection-efficiency data. Results demonstrate good consistency with the Eulerian formulation and agreement with available experimental data.

3.1 Introduction

Particle-laden flows have long been a subject of scientific investigation due to their prevalence in numerous applications ranging from climate prediction [121] to combustion systems [122] and, in the context of the present work, aircraft ice accretion [61, 123–125]. A key challenge lies in the large number of particles required for high-fidelity simulations, which imposes significant computational demands. With recent advances in high-performance computing (HPC) on distributed systems, there is a growing need to optimize computational frameworks for tracking large particle populations.

Recent Ice Prediction Workshops [27, 28] have shown an increasing use of Eulerian droplet-tracking methods [126–130]. These methods adopt the same computational framework as the flow solver, providing advantages in terms of implementation and computational efficiency. However, Eulerian formulations can present challenges when incorporating complex droplet phenomena such as splashing and secondary-droplet re-impingement. Lagrangian formulations, on the other hand, provide direct tracking of individual droplet trajectories but can become computationally expensive when a large number of droplets is required. Advances in high-performance computing therefore provide an opportunity to improve the computational efficiency of Lagrangian approaches for aircraft icing applications.

Significant progress has been made in the development of Lagrangian particle-tracking methods for ice accretion, with recent studies proposing different strategies to address their

computational challenges. For instance, [124] adapts the grid-partitioning scheme based on particle trajectories, while [125] employs a dynamic load-balancing approach in which a dedicated computational rank redistributes droplets across the remaining ranks. Further developments are nevertheless required to reduce oscillations in surface collection-efficiency distributions [131] and to automate the generation and refinement of the droplet-seeding grid. The present work addresses these aspects by introducing improvements throughout the Lagrangian tracking process, from droplet seeding to trajectory integration, geometric intersection, surface impingement, and parallel execution. The chapter is structured as follows: Section 3.2 reviews droplet-tracking techniques for ice accretion applications and introduces the Lagrangian formulation used in this work. Section 3.3 presents the proposed solver enhancements. Section 3.4 presents and discusses the benchmark cases and corresponding results. Finally, Section 3.5 summarizes the main findings.

3.2 Lagrangian Droplet Tracking for Ice Accretion

Water droplet tracking is a key component of ice accretion simulations, as it determines both impingement locations and the amount of water collected on solid surfaces. Numerous approaches have been proposed in the literature, all aiming to balance robustness, accuracy, and computational efficiency.

Historically, early ice accretion solvers such as LEWICE [132] relied on Lagrangian tracking, in which individual droplet trajectories are computed by solving their equations of motion. While well suited to panel-based potential-flow solvers, the extension of this approach to modern RANS-based frameworks introduces additional complexity due to the need for repeated interpolation of flow quantities at droplet locations.

To address these limitations, Eulerian approaches [56] were introduced, modeling droplets as a continuous dispersed phase governed by transport equations for momentum and volume fraction α (Fig. 3.1). This formulation integrates naturally within flow solvers and offers excellent computational efficiency and scalability. However, accurately representing complex droplet physics, such as splashing, remains challenging and often computationally expensive. These limitations motivate the consideration of Lagrangian particle tracking, in which individual droplet trajectories and post-impact behavior can be treated more directly.

Lagrangian droplet tracking is based on integrating the equation of motion from Newton's second law (Eq. 3.1) for each droplet individually. In the present formulation, the forces considered are the drag force resulting from the relative motion between the air and the droplet, and the gravitational force. The droplet velocity $\boldsymbol{u_d}$ is obtained from Eq. 3.1, where

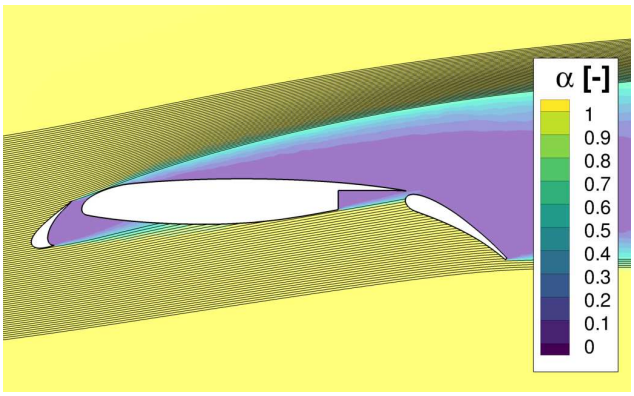


Figure 3.1 Example of Eulerian solver results : Countour of water volume fraction α with droplet trajectories streamlines.

$\boldsymbol{u_a}$ is the air velocity. The droplet position $\boldsymbol{x_d}$ is subsequently obtained from Eq. 3.2. The force coefficient F is defined by Eq. 3.3 as a function of the droplet drag coefficient C_D, the relative velocity $\|\boldsymbol{u_a} - \boldsymbol{u_d}\|$, the air and droplet densities ρ_a and ρ_d, and the droplet diameter D.

$$\frac{d\boldsymbol{u_d}}{dt} = \frac{\mathbf{f}}{m} + \mathbf{g} = F(\boldsymbol{u_a} - \boldsymbol{u_d}) + \mathbf{g} \tag{3.1}$$

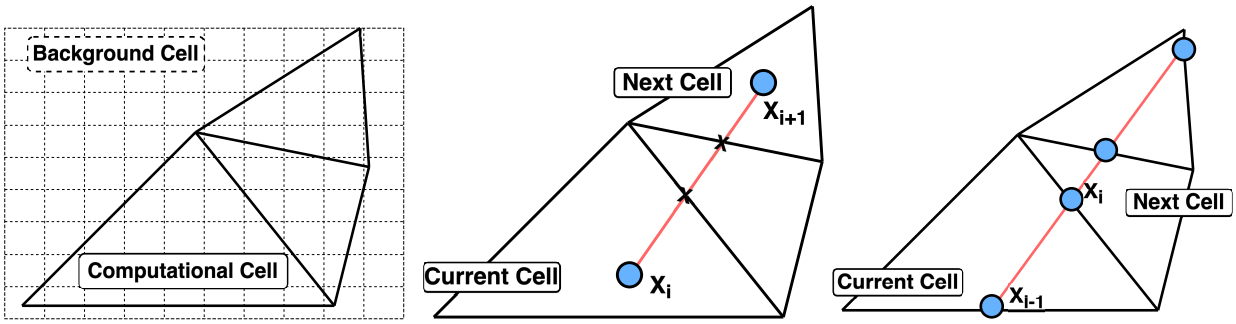
$$\frac{d\boldsymbol{x_d}}{dt} = \boldsymbol{u_d} \tag{3.2}$$

$$F = \frac{3C_D\rho_a\|\boldsymbol{u_a} - \boldsymbol{u_d}\|}{4\rho_d D} \tag{3.3}$$

The main assumptions of the formulation are the following:

- The droplet mass remains constant prior to impact;
- Droplets are independent from one another, such that collisions and coalescence are neglected;
- The droplet concentration does not affect the fluid flow;
- The effects of turbulence on the droplets are neglected;
- Droplet rotation and lift are neglected;
- Phase change within the droplets is not considered, as the solver is limited to computing the deposited liquid water mass under the quasi-steady icing hypothesis [133].

A central difficulty in Lagrangian particle tracking on unstructured meshes is that droplet trajectories do not generally follow the computational grid. The integration of Eq. 3.1 requires the interpolation of the flow velocity along the trajectory and the identification of the cells traversed by the droplet. Various approaches have been developed for this purpose. Tree-based search techniques [134] and auxiliary-grid approaches [135], illustrated in Fig. 3.2a, rely on spatial-search structures to identify the host cell. Alternatively, face-to-face tracking techniques use the connectivity of the computational mesh to follow the droplet from one cell to the next.



(a) Auxiliary grid tracking [135].

(b) face-to-face tracking [61] (In Cell Positions).

(c) face-to-face tracking [123] (On Face Positions).

Figure 3.2 Comparison of droplet tracking techniques used to identify the computational cells traversed by a droplet trajectory: (a) auxiliary-grid tracking, (b) face-to-face tracking with droplet positions located inside the computational cells, and (c) face-to-face tracking with droplet positions located on the intersected cell faces.

The face-to-face approach of [61], illustrated in Fig. 3.2b, uses the previous droplet position and velocity together with the mesh connectivity to identify the successive cells traversed by the trajectory. This approach requires particular care near solid boundaries, where strong flow gradients and small cells can affect the trajectory integration and impact detection.

The face-to-face tracking method presented by [123], illustrated in Fig. 3.2c, instead advances the droplet from one mesh face to the next using edge-to-cell connectivity in 2D or face-to-cell connectivity in 3D. A ray is cast based on the droplet velocity, and the nearest face intersection determines the next cell along the trajectory. The corresponding residence time within the cell defines the local time step, which therefore adapts to the local mesh resolution. The host cell is obtained directly from the mesh connectivity without requiring an additional spatial search. The method also operates directly on the same unstructured mesh used by the flow solver. For these reasons, this face-to-face formulation is adopted in the present work.

The integration of Eq. 3.1 is achieved using a second order backward difference scheme [123]:

$$a\boldsymbol{u_d}^n + b\boldsymbol{u_d}^{n-1} + c\boldsymbol{u_d}^{n-2} = F(\boldsymbol{u_a} - \boldsymbol{u_d}) + \mathbf{g} \tag{3.4}$$

$$\boldsymbol{u_d}^n = \frac{F\boldsymbol{u_a} - b\boldsymbol{u_d}^{n-1} - c\boldsymbol{u_d}^{n-2} + \mathbf{g}}{a + F} \tag{3.5}$$

$$a = \frac{2\Delta t_c + \Delta t_{c-1}}{(\Delta t_c + \Delta t_{c-1})\Delta t_c} \tag{3.6}$$

$$b = \frac{-(\Delta t_c + \Delta t_{c-1})}{\Delta t_c \Delta t_{c-1}} \tag{3.7}$$

$$c = \frac{\Delta t_c}{(\Delta t_c + \Delta t_{c-1})\Delta t_{c-1}} \tag{3.8}$$

where Δt_c and Δt_{c-1} are the current and previous local time steps, respectively, and the superscripts n, $n-1$, and $n-2$ denote successive integration levels. Since the local time step depends on the next face intersection, the droplet velocity and intersection position are determined iteratively. The flow velocity at the face $\boldsymbol{u_a}^n$ is first evaluated by averaging the flow velocity between the current and next cells:

$$\boldsymbol{u_a}^n = \frac{1}{2}(\boldsymbol{u_{ac}} + \boldsymbol{u_{ac+1}}) \tag{3.9}$$

Since the next cell depends on the trajectory itself, an iterative procedure is employed. Initially, $\boldsymbol{u_d}^n$ is assumed equal to $\boldsymbol{u_d}^{n-1}$ and the nearest face intersection is determined. The corresponding local time step and the coefficients in Eqs. 3.6–3.8 are then calculated. Equation 3.5 provides an updated value of $\boldsymbol{u_d}^n$, and the procedure is repeated until the prescribed convergence criterion is satisfied. The face intersection is determined using the intermediate droplet velocity

$$\boldsymbol{u_d}^{n-\frac{1}{2}} = \frac{1}{2}(\boldsymbol{u_d}^n + \boldsymbol{u_d}^{n-1}) \tag{3.10}$$

In regions of strong flow gradients, particularly near solid surfaces, the method can locally reduce to first-order accuracy [123]. To improve the evaluation of the flow velocity at the crossed face, the cell-centered velocity is reconstructed at the facet-intersection position using the local velocity gradient:

$$\boldsymbol{u}_{\boldsymbol{a},face} = \boldsymbol{u}_{\boldsymbol{a},c} + \nabla(\boldsymbol{u}_{\boldsymbol{a}})_c \cdot \boldsymbol{d} \tag{3.11}$$

where $\boldsymbol{d}$ is the displacement vector from the cell center to the facet-intersection position.

3.3 Improvements in Lagrangian Particle Tracking

Several improvements are introduced to enhance the robustness, accuracy, and computational efficiency of the Lagrangian particle-tracking methodology, including trajectory intersection, adaptive seeding, collection-efficiency evaluation, SLD treatment, and parallel workload distribution.

3.3.1 Ray–Quadrilateral Intersections in CFD Applications

Hexahedral elements are commonly used in CFD meshes because they can provide high-quality and structured resolution in regions of interest. This is illustrated in Fig. 3.3 using the Common Research Model (CRM) wing–body configuration from the Fifth AIAA High-Lift Prediction Workshop (HLPW5), where successive levels of hexahedral refinement are used around the aircraft geometry. For Lagrangian particle tracking on such meshes, the trajectory calculation repeatedly requires the intersection of the droplet path with the quadrilateral faces of the hexahedral elements.

Unlike triangular faces, quadrilateral faces are not necessarily planar. In a general unstructured hexahedral mesh, the four vertices defining a face may therefore not lie on a single plane, making a planar ray–face intersection ambiguous. A common approach is to split the quadrilateral into two planar triangles and perform two independent ray–triangle intersection tests following the method of [136]. Although straightforward, this introduces an artificial diagonal and treats the quadrilateral as two separate planar surfaces.

An alternative is to represent the quadrilateral directly as a bilinear patch, parameterized by the local coordinates $(u, v) \in [0, 1]^2$. The particle trajectory is represented by the ray $\boldsymbol{r}(t)$,

$$\boldsymbol{r}(t) = \boldsymbol{O} + t\boldsymbol{D}, \quad t \geq 0 \tag{3.12}$$

while the quadrilateral is represented by the bilinear patch

$$\boldsymbol{p}(u, v) = (1-u)(1-v)\boldsymbol{p}_{00} + (1-u)v\boldsymbol{p}_{01} + u(1-v)\boldsymbol{p}_{10} + uv\boldsymbol{p}_{11} \tag{3.13}$$

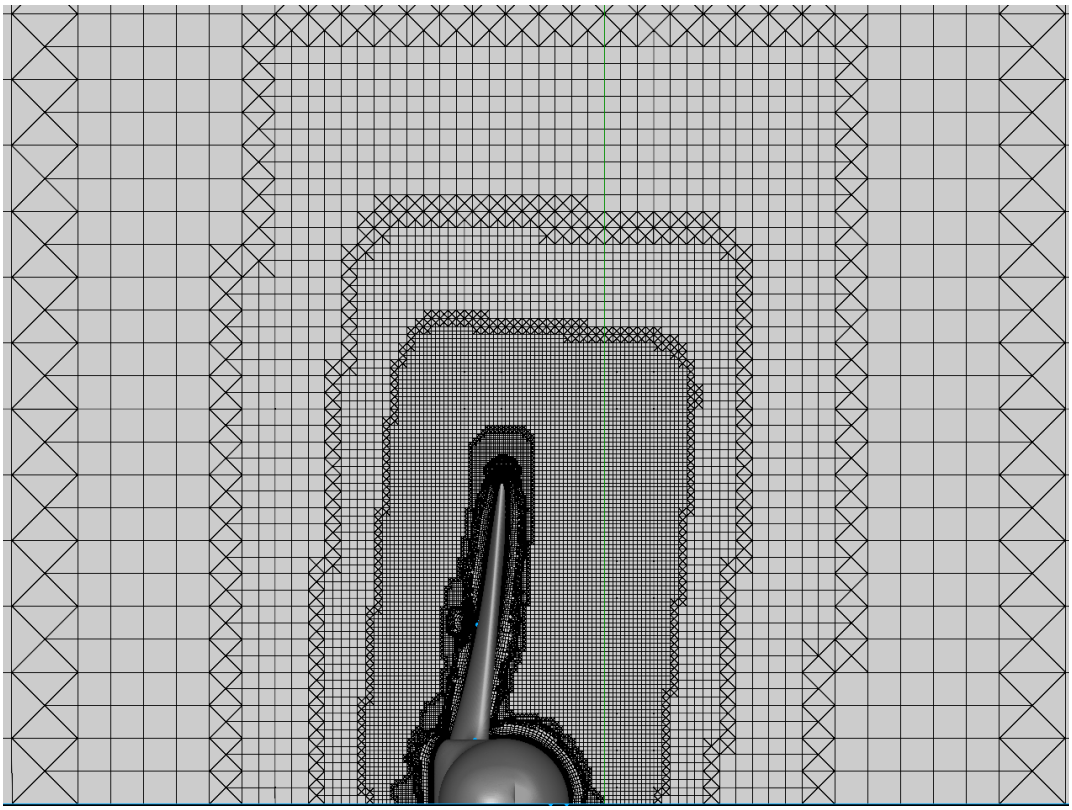
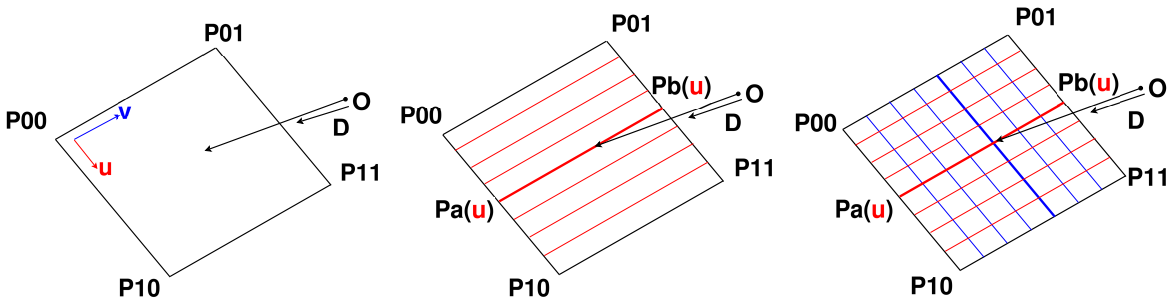


Figure 3.3 Cut through the computational mesh of the CRM wing–body configuration from the Fifth AIAA High-Lift Prediction Workshop (HLPW5), illustrating the hexahedral refinement around the aircraft geometry.

Here, $\boldsymbol{O}$ and $\boldsymbol{D}$ denote the ray origin and direction, respectively, and $\boldsymbol{p}_{ij}$ are the four vertices of the quadrilateral. This representation defines a continuous surface passing through the four vertices without requiring them to be coplanar. The ray–quadrilateral intersection problem therefore consists of determining the parameters (t, u, v) for which $\boldsymbol{r}(t) = \boldsymbol{p}(u, v)$.

Several algorithms have been proposed for computing ray–bilinear patch intersections. Equating Eqs. 3.12 and 3.13 yields a system of three equations that can be solved analytically [137]. This algebraic formulation has been used in Lagrangian particle-tracking applications [138, 139]. The more recent GARP (Geometric Approach to Ray/bilinear Patch intersections) method of [140] instead uses the geometric properties of the bilinear patch to determine the intersection.

The GARP method interprets the bilinear patch as a ruled surface obtained by linear interpolation between two opposite edges. The intersection procedure is carried out in two parametric stages, as illustrated in Fig. 3.4b–3.4c. For a given value of u, the corresponding ruling segment is defined by the two interpolated points



(a) Step 0: Definition of the ray and bilinear patch.

(b) Step 1: Determination of the intersection parameter u.

(c) Step 2: Determination of the intersection parameter v.

Figure 3.4 Geometrical construction used for the ray–bilinear patch intersection, showing the definition of the ray and patch geometry followed by the successive determination of the intersection parameters u and v.

$$\boldsymbol{P}_a(u) = (1-u)\boldsymbol{p}_{00} + u\boldsymbol{p}_{10}, \qquad \boldsymbol{P}_b(u) = (1-u)\boldsymbol{p}_{01} + u\boldsymbol{p}_{11} \tag{3.14}$$

The first stage determines the values of u for which this segment is coplanar with the ray. This condition is expressed by setting the scalar triple product of the vectors $\boldsymbol{OP}_a$, $\boldsymbol{P}_a\boldsymbol{P}_b$, and $\boldsymbol{D}$ to zero, leading to a polynomial equation of degree at most two in u. Only solutions satisfying $u \in [0,1]$ are retained.

For each retained value of u, the ruling segment $\boldsymbol{P}_a\boldsymbol{P}_b$ is therefore fixed. The second stage determines v and t by finding the closest points between the line supporting this segment and the line supporting the ray. A valid intersection requires $v \in [0,1]$ and $t \geq 0$. In the presence of multiple valid solutions, the intersection corresponding to the smallest admissible value of t is selected.

Whereas Ramsey et al. [137] determine the intersection by algebraically manipulating the ray and bilinear-patch equations (Eqs. (3.12) and (3.13)), resulting in relatively complex intermediate expressions. GARP instead exploits the geometric nature of the problem to obtain a coplanarity condition for one patch parameter. Both approaches lead to a polynomial equation of degree at most two, but differ in how the intersection parameters are determined. The geometric formulation of GARP reduces the number of arithmetic operations required to determine the intersection [140].

To evaluate computational performance, one million random rays and quadrilateral patches were generated for intersection testing using the GARP and Ramsey et al. [137] formulations, together with the traditional two-triangle approach. Both ray–patch methods reduce the

computational cost relative to the two-triangle approach, with GARP providing an additional improvement over the formulation of Ramsey et al. [137]. The implications of the intersection algorithm on the overall solver scalability are discussed in Section 3.4.

Table 3.1 Performance comparison of ray–quadrilateral intersection methods.

Method	ETR* vs. Two-Triangles [136]	ETR* vs. Ramsey et al. [137]
Ramsey et al. [137]	51.3%	-
GARP [140]	58.2%	14.2%

* ETR: execution-time reduction.

3.3.2 Adaptive Seeding

The definition of the seeding region is an important step in Lagrangian droplet tracking, as the initial droplet seeding distribution must adequately cover the portion of the incoming cloud that can impinge on the geometry. A first estimate can be obtained by projecting the geometry onto an upstream seeding plane along the freestream direction. An initial set of trajectories can then be computed, and the droplets that effectively impact the geometry are used to adjust the dimensions of the seeding region.

A more systematic estimation of the seeding region can be obtained through trajectory backtracking. Direct backtracking from the aerodynamic RANS solution is not possible because the no-slip boundary condition imposes a zero fluid velocity at the wall. The Eulerian droplet formulation provides a complementary solution to this problem, since the computed droplet velocity remains defined at the impingement location. Building on the 2D hybrid approach of [5], the Eulerian droplet velocity field is therefore used to trace trajectories backward from the surface toward a user-defined upstream seeding plane, as illustrated in Fig. 3.5. The resulting intersections with the seeding plane provide an estimate of the region from which impacting droplets originate and are used to initialize the Lagrangian seeding grid.

Once the initial seeding region has been established, the spatial resolution of the seeding grid becomes the main factor controlling convergence of the Lagrangian impingement solution. Rather than prescribing a uniformly fine distribution of droplets, an adaptive refinement procedure is employed. An initial structured seeding grid is generated and the corresponding droplet trajectories are computed. Cells of the seeding grid associated with droplets that impact the geometry are subsequently refined, thereby progressively increasing the number of trajectories in regions contributing to the impingement solution while avoiding unnecessary refinement elsewhere.

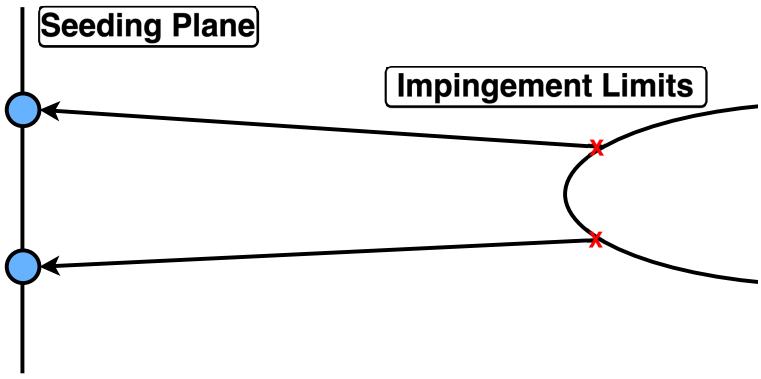


Figure 3.5 Hybrid Eulerian–Lagrangian backtracking for initialization of the Lagrangian seeding region [5].

Figure 3.6 illustrates this process. Starting from a relatively coarse initial distribution, successive refinement levels concentrate the seeding resolution within the region associated with impacting trajectories. Previously computed droplets are retained between refinement passes, such that only the newly introduced seeding points require additional trajectory calculations. In practice, approximately six to eight refinement passes are typically sufficient, although the refinement can instead be continued until the variation in the total collection efficiency criterion between successive refinement passes falls below a prescribed tolerance of $\epsilon = 10^{-5}$. Another advantage of performing the refinement over multiple passes is that the workload distribution can be progressively improved from one pass to the next. The computational time of previously computed trajectories is retained and used to estimate the workload associated with newly generated droplets during subsequent refinement passes.

3.3.3 Collection Efficiency Computations and SLD Effects

The collection efficiency associated with a Lagrangian trajectory is commonly evaluated by constructing a local droplet streamtube around the injected droplet, as illustrated in Fig. 3.7a. In principle, this droplet streamtube can be constructed directly from neighboring droplets of the seeding grid. However, there is no guarantee that neighboring droplets upstream will impact the same region of the geometry. This becomes particularly problematic for complex configurations, where neighboring trajectories may separate and impact different geometric components, as illustrated in Fig. 3.7b. A sufficiently small spacing between droplets can reduce the occurrence of such situations, but applying this resolution to the complete seeding grid can lead to a prohibitively large number of trajectories.

To avoid this requirement, the droplets originating from the adaptive seeding grid are treated

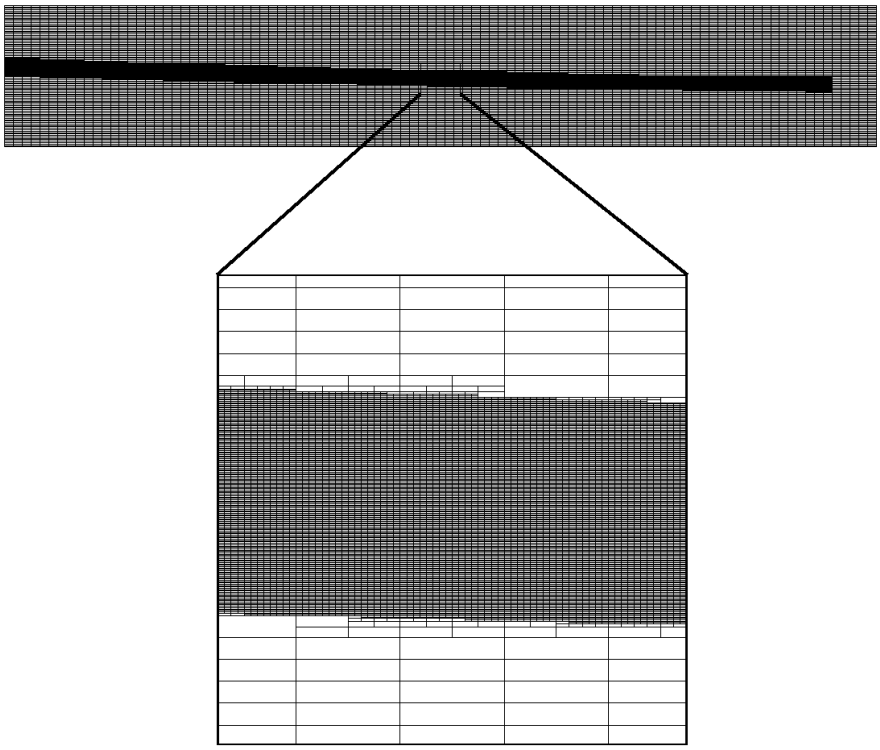


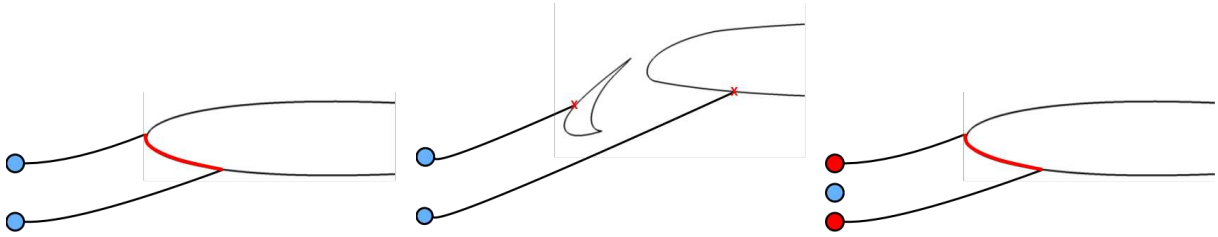
Figure 3.6 Example of adaptive refinement of the Lagrangian seeding grid around the region associated with impacting droplet trajectories.

as primary droplets, while additional auxiliary droplets are introduced locally for the sole purpose of constructing the streamtube and evaluating the collection efficiency [141]. For each primary droplet, two auxiliary droplets are released in its immediate vicinity in 2D, while three or four auxiliary droplets are used in 3D depending on the topology of the local streamtube. The auxiliary droplets are initialized with a spacing of $dy \approx O(10^{-8}c_{\text{ref}})$ from the primary droplet, where c_{ref} is the reference length of the geometry, as illustrated in Fig. 3.7c. The resulting local streamtube is then used to determine the collection efficiency associated with the primary impact as

$$\beta = \frac{A_\infty}{A_s} \tag{3.15}$$

where A_∞ is the area of the local streamtube at the upstream seeding location and A_s is the

corresponding area at the surface impact location. The auxiliary droplets are used only for this calculation and are discarded afterward.



(a) Conventional droplet streamtube. (b) Pathological streamtube configuration. (c) Local streamtube with auxiliary droplets.

Figure 3.7 Collection-efficiency computation based on droplet streamtubes: (a) conventional streamtube defined by neighboring primary droplets, (b) pathological configuration leading to an ill-defined streamtube, and (c) local streamtube constructed using auxiliary droplets around the primary trajectory. Primary and auxiliary droplets are represented in blue and red, respectively.

Once the collection efficiency has been evaluated at each primary impact location, the Lagrangian information is transferred to the cell-centered surface representation used in CHAMPS. Since multiple primary droplets may contribute to the same surface face, a distance-weighted average is applied to obtain a single value β_i for each impacted surface cell:

$$\beta_i = \frac{\sum \beta_{i,d} \cdot \omega_{i,d}}{\sum \omega_{i,d}}, \tag{3.16}$$

where the weight $\omega_{i,d}$ is based on the inverse square of the distance between the primary-droplet impact position and the center of the corresponding surface face. To improve the surface coverage without increasing the number of primary trajectories, the influence of each impact is extended to neighboring boundary faces identified through the existing surface connectivity. This treatment extends the interpolation of the available Lagrangian information to neighboring faces without requiring additional trajectory calculations.

Experimentally, an icing cloud does not consist of droplets of a single diameter, but rather of a distribution of droplet sizes, each representing a fraction of the total liquid water content. To better represent these experimental conditions, a polydisperse cloud is discretized into a number of droplet-size bins, and the preceding procedure is performed independently for each bin. Each bin n is assigned a weighting factor γ_n corresponding to its fraction of the distribution, and the resulting collection efficiencies are combined to obtain the total

collection efficiency on each surface cell:

$$\beta_{i,total} = \sum_{n=1}^{N_{bins}} \gamma_n \beta_{i,n}, \tag{3.17}$$

where $\beta_{i,n}$ is the collection efficiency on surface cell i for droplet-size bin n. This formulation allows the contribution of the complete droplet-size distribution to be represented within the final surface collection-efficiency.

For SLD conditions, the models introduced in Section 2.3.3 are subsequently applied to account for the fraction of the impinging water that is not retained following impact. The mass-loss coefficient f_m, or equivalently the sticking rate $\epsilon = 1 - f_m$, modifies the collection efficiency associated with the primary impact according to

$$\beta_{i,\,final} = (1 - f_m)\,\beta_{i,\,0} = \epsilon\,\beta_{i,\,0}, \tag{3.18}$$

where $\beta_{i,\,0}$ denotes the collection efficiency prior to the splashing correction. In addition to this correction, the model of Wright et al. [65] provides the post-impact properties required to track the secondary droplets generated by splashing. In the present implementation, as a simplification, the ejected mass associated with each impact is represented by a single secondary-droplet parcel rather than by individual secondary droplets. The post-impact normal and tangential velocities and the secondary-droplet diameter are given by

$$\begin{aligned} u_{n,\,post\,impact} &= u_{n,\,impingement}(1.075 - 0.0025\,\alpha_{impingement}) \\ u_{t,\,post\,impact} &= u_{t,\,impingement}(0.300 - 0.0020\,\alpha_{impingement}) \end{aligned} \tag{3.19}$$

$$\frac{D_{new}}{D} = exp(-0.0281\,K_M). \tag{3.20}$$

The secondary-droplet parcel is then tracked from the primary impact location until it either leaves the computational domain or encounters the geometry again, as illustrated in Fig. 3.8. In the case of re-impingement, its contribution to the local collection efficiency is normalized with respect to the incoming water flux according to

$$\beta_{Re-Impingement} = \frac{m_{sd} \cdot (\boldsymbol{u_{sd}} \cdot \boldsymbol{n})/A_s}{m_\infty \cdot u_\infty / A_\infty} \tag{3.21}$$

where m_{sd} is the mass contained in the secondary-droplet parcel, $\boldsymbol{u_{sd}}$ is its velocity at the

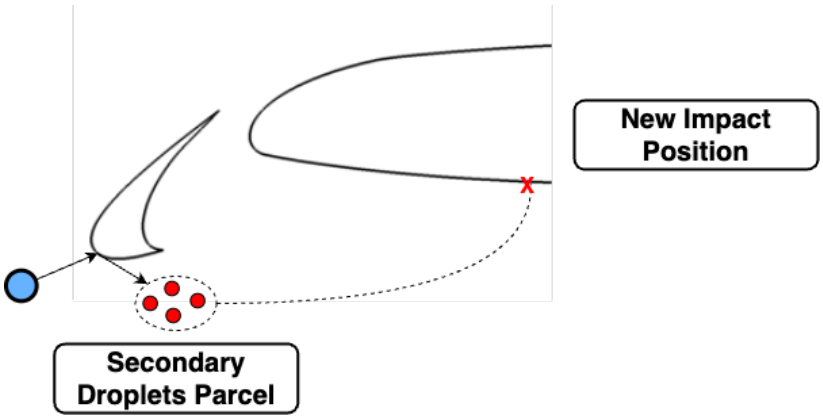


Figure 3.8 Schematic representation of secondary-droplet parcel tracking and re-impingement following splashing.

re-impingement location, A_s is the area of the impacted surface face, A_∞ is the area of the upstream seeding grid, and m_∞ is the initially injected droplet mass. This contribution is added to the collection efficiency of the corresponding boundary face to account for secondary-droplet re-impingement.

3.3.4 Parallelism

The Lagrangian droplet solver is implemented in Chapel within the distributed-memory framework of CHAMPS [142]. The aerodynamic mesh remains spatially decomposed among the available computational resources, whereas the adaptive droplet cloud is generated and scheduled independently of this decomposition. This separation is particularly useful for Lagrangian tracking, since the computational cost of a trajectory is not necessarily correlated with the spatial partition in which its seeding point is located. To minimize inter-node communication during trajectory integration, the flow and mesh data required for droplet tracking are replicated across the computational locales. This allows trajectories to access the required aerodynamic and geometric information locally, avoiding repeated remote data accesses as droplets travel across the computational domain. Each computational resource is therefore assigned a subset of the droplet population and independently tracks these droplets through the domain until impact or trajectory termination.

The adaptive seeding procedure progressively introduces new trajectories during refinement, each of which is assigned an estimated computational cost for workload distribution. During the initial refinement stage, all trajectories are assigned the same cost. In subsequent stages, the cost of each new trajectory is estimated from the measured cost of previously computed trajectories within its parent cell, progressively improving the workload estimate throughout

the refinement process.

The resulting trajectories are distributed using a longest-processing-time (LPT) scheduling strategy. Trajectories are first sorted according to their predicted computational cost and are successively assigned to the computational resource with the lowest accumulated predicted workload. After integration, their measured computational costs are retained and used to improve the workload estimates during subsequent refinement stages. Previously computed trajectories are also cached and reused, such that only newly introduced droplets contribute to the computational workload during refinement.

Auxiliary-droplet calculations associated with wall impacts are balanced separately from the primary trajectories. Following the primary-droplet calculation, impacting droplets are redistributed using a second cost-based assignment before auxiliary-droplet integration. The measured cost of each primary trajectory, together with the number of auxiliary droplets required to construct the local streamtube, is used to estimate this additional workload. This two-stage scheduling procedure allows both primary and auxiliary trajectory calculations to be distributed according to their expected computational cost rather than solely according to the number of droplets.

3.4 Results

The results are first presented through a parallel scalability study to evaluate the computational performance of the Lagrangian solver on distributed-memory systems. The IPW2 CRM Hybrid configuration is used for this scalability assessment. The droplet-tracking methodology and SLD implementation are then assessed using the two IPW1 configurations summarized in Table 3.2, for which collection-efficiency results are compared with available numerical and experimental data. Finally, the HLPW5 CRM wing–body configuration is considered as a large 3D application of the Lagrangian methodology.

3.4.1 Scalability

To evaluate the parallel scalability of the Lagrangian solver, Case IPW2 - 1.3 is selected as the benchmark. All simulations are conducted on the Digital Research Alliance of Canada Trillium cluster. Each compute node contains two AMD EPYC 9655 processors, providing a total of 192 cores per node. Simulations are performed using between 2 and 16 nodes, with the number of nodes doubled at each measurement point.

The flow conditions from the original workshop case are adopted. A single droplet diameter corresponding to the median volumetric diameter (MVD) of 25 μm is considered. A total of

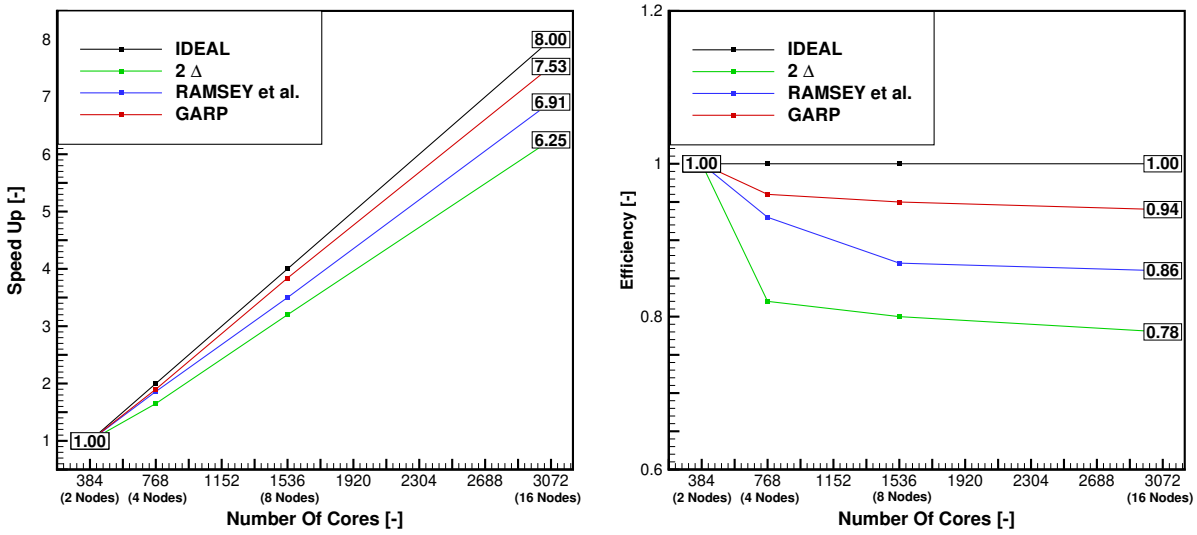
Table 3.2 Summary of the droplet-impingement benchmark cases.

Parameter	IPW2-1.3	IPW1-112	IPW1-122	HLPW5-2.1
Geometry	CRM Hybrid	NACA 64A008 H-tail	Three-element airfoil	CRM wing–body
MVD $[\mu m]$	25	92	92	FR*
AOA [°]	3.7	6.0	4.0	6.0
Ma [-]	0.21	0.23	0.23	0.20
C_{ref} $[inch]$	39.37	37.65	36.0	275.8
T $[K]$	247.15	280.0	278.0	288.15
Re [-]	$8.1 \cdot 10^6$	$5.0 \cdot 10^6$	$4.8 \cdot 10^6$	$5.4 \cdot 10^6$

Note: *FR: Freezing Rain.

12.8 million droplets are seeded for the benchmark, providing a sufficiently large and fixed workload for the scalability assessment.

Unlike Eulerian simulations, for which the computational cost can conveniently be characterized by the time per solver iteration, individual Lagrangian trajectories may require substantially different numbers of integration steps before impacting the geometry or leaving the computational domain. Consequently, the total computation time is used as the performance metric for the scalability study.



(a) Solver speedup.

(b) Solver efficiency.

Figure 3.9 Parallel scalability of the Lagrangian solver: speedup and parallel efficiency relative to the two-node baseline.

The computational speedup is determined using Eq. 3.22, and the parallel efficiency is com-

puted using Eq. 3.23. Since the two-node calculation is used as the baseline, a factor of 2 is included in the efficiency definition. The results are presented in Fig. 3.9. With the GARP technique, the solver retains approximately 94% parallel efficiency at 16 nodes relative to the two-node baseline. Over the considered range, GARP also exhibits better scalability than the two other approaches.

The remaining loss in parallel efficiency can be associated with several factors inherent to Lagrangian particle tracking. Although the flow and mesh data required for trajectory integration are replicated to minimize communication between computational resources, droplet trajectories access these data according to their paths through the domain, resulting in irregular memory-access patterns. In addition, the high-level parallel abstractions provided by Chapel facilitate the implementation of distributed-memory algorithms but provide less direct control over some architecture-specific, low-level optimizations. These effects become increasingly relevant as the computational workload is distributed over a larger number of resources and may contribute to the departure from ideal parallel scaling.

$$Speed\,Up = \frac{Simulation\,Time\,2\,Nodes}{Simulation\,Time\,N\,Nodes} \tag{3.22}$$

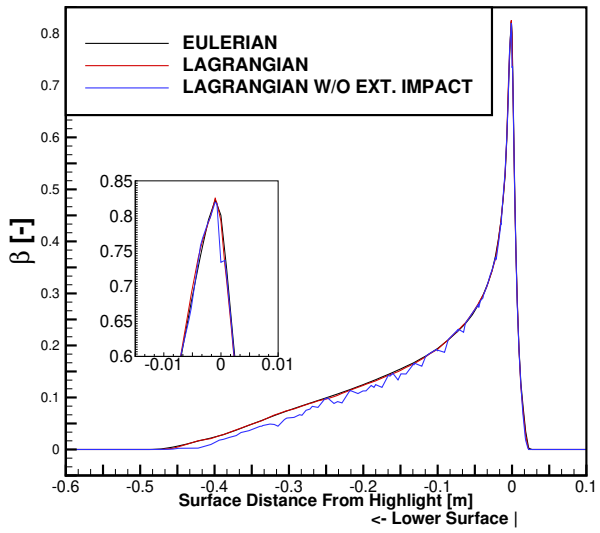
$$\eta = 2 \cdot \frac{Speed\,Up}{N\,Nodes} \tag{3.23}$$

3.4.2 Case IPW1 - 112: NACA64A008 Horizontal Tail

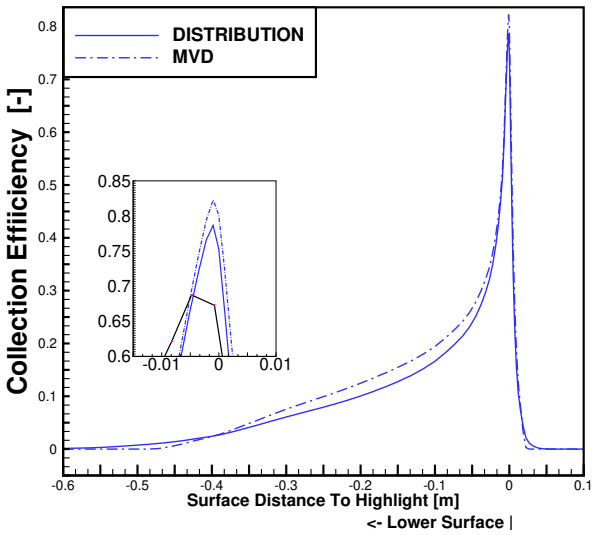
Case IPW1-112 corresponds to a NACA64A008 horizontal-tail configuration considered during IPW1, for which experimental collection-efficiency measurements are available under SLD conditions. Figure 3.10 presents the corresponding collection-efficiency results. The effect of extending the influence of each droplet impact to neighboring surface faces is first illustrated in Fig. 3.10a. Without this treatment, the Lagrangian solution exhibits local oscillations in the collection-efficiency distribution. Extending the interpolation to neighboring faces produces a smoother distribution and improves the consistency with the Eulerian prediction.

The influence of the droplet-size distribution is then shown in Fig. 3.10b. Compared with the single-diameter calculation using the MVD of 92μm, the polydisperse case modifies the collection-efficiency distribution. The different droplet diameters respond differently to the aerodynamic flow field, resulting in broader impingement limits.

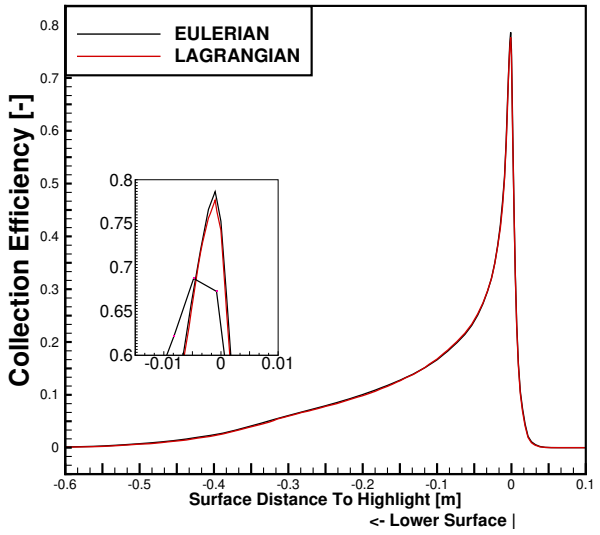
Figure 3.10c compares the Eulerian and Lagrangian formulations using the droplet-size distribution prescribed by the workshop. Close agreement is obtained between the two formulations over the complete impingement region.



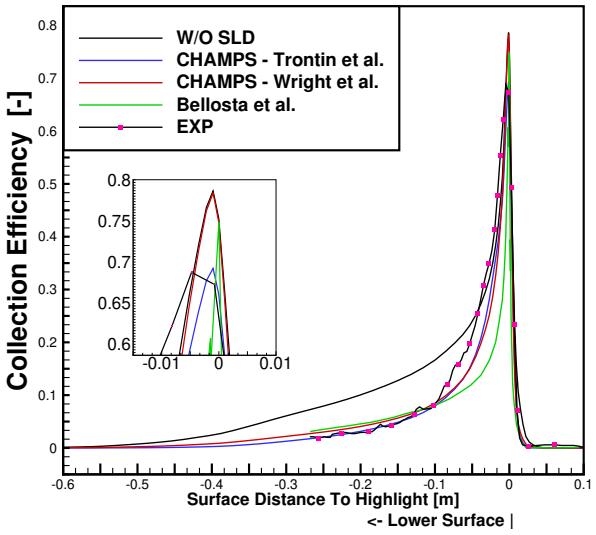
(a) Effect of droplet impact zone.



(b) Effect of droplet distribution on collection efficiency.



(c) Impingement results.



(d) SLD results.

Figure 3.10 Collection-efficiency results for Case IPW1-112: impact-zone treatment, droplet-size distribution, Eulerian–Lagrangian comparison, and SLD-model assessment.

Finally, the influence of SLD effects is evaluated in Fig. 3.10d through comparison with the experimental collection-efficiency data of [7]. Without SLD effects, the predicted collection efficiency is substantially higher than the experimental measurements. Accounting for post-impact mass loss reduces the retained water and improves agreement with the experimental distribution.

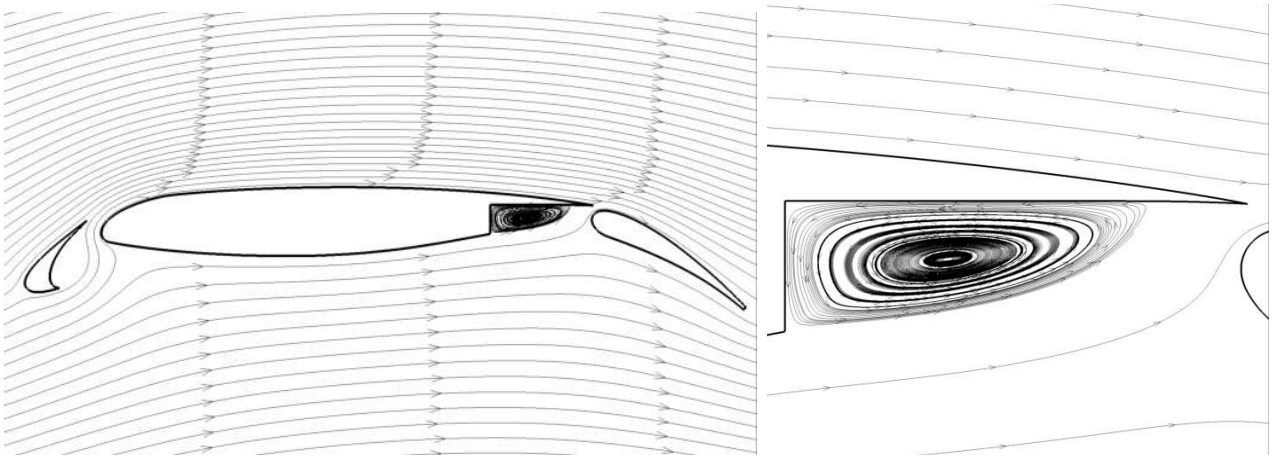
The model of Wright et al. [65] predicts little mass loss near the stagnation region, where the droplets impact the surface nearly normally. Consequently, the collection efficiency remains close to the solution without SLD effects near the stagnation point. As the impact becomes increasingly tangential farther downstream, the predicted mass loss increases and progressively reduces the collection efficiency. In contrast, the model of Trontin et al. [69] predicts mass loss near the stagnation region as a function of the impact conditions, resulting in a lower peak collection efficiency and closer agreement with the experimental measurements. Both models progressively reduce the collection efficiency away from the stagnation region and improve agreement with the experimental distribution.

The CHAMPS predictions are also compared with the results reported by [61], which employ the SLD model of Wright et al. [65]. For this case, CHAMPS provides closer agreement with the experimental collection-efficiency measurements. An important methodological difference lies in the evaluation of the collection efficiency from the computed trajectories. The local streamtube approach employed in the present methodology may contribute to the improved agreement obtained with CHAMPS.

3.4.3 Case IPW1 - 122: Three Element Airfoil

Case IPW1-122 also corresponds to a configuration for which experimental collection-efficiency measurements are available under SLD conditions. The three-element airfoil presents a particularly challenging impingement problem because of the recirculation region in the cove behind the main element, illustrated in Fig. 3.11. Disparities in the predicted collection efficiency were observed among the IPW1 participants, particularly on the main and flap components (Fig. 3.12). Figures 3.13a–3.13c show the CHAMPS Eulerian and Lagrangian results using the droplet-size distribution prescribed by the workshop. Overall, the two formulations exhibit good agreement. The Eulerian formulation predicts some impingement toward the end of the main element on both sides of the geometry. This behavior is associated with non-physical values of the droplet volume fraction resulting from dispersion-induced oscillations. The slat collects the largest fraction of the incoming droplets because of its upstream position. This effect is particularly pronounced on the pressure side at the prescribed angle of attack of 4°. Consequently, substantially less impingement is observed on the suction

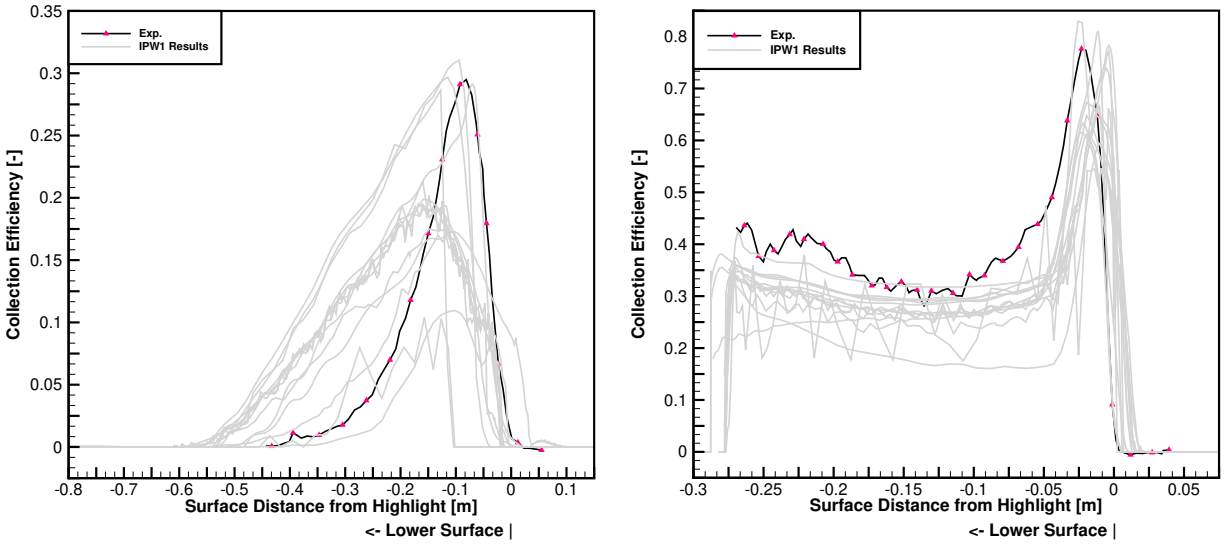
sides of the main and flap elements. The remaining differences between the Eulerian and Lagrangian predictions may be related to numerical dissipation in the Eulerian formulation, particularly near the cove behind the main element, as discussed in [59]. These differences are expected to decrease with mesh refinement, as illustrated in Appendix B.1.



(a) Three Element Airfoil Geometry.

(b) Recirculation zone in the cove of the main element.

Figure 3.11 Description of Case IPW1 - 122.



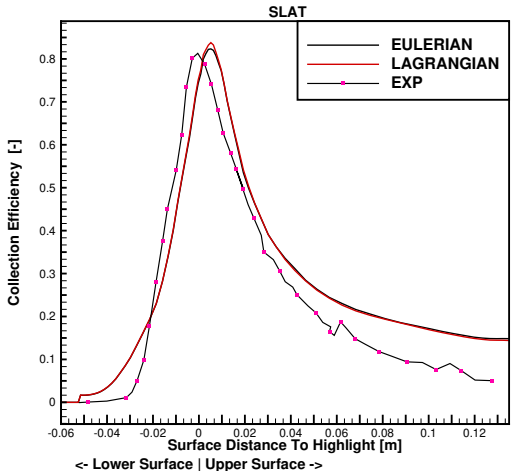
(a) Main component results.

(b) Flap component results.

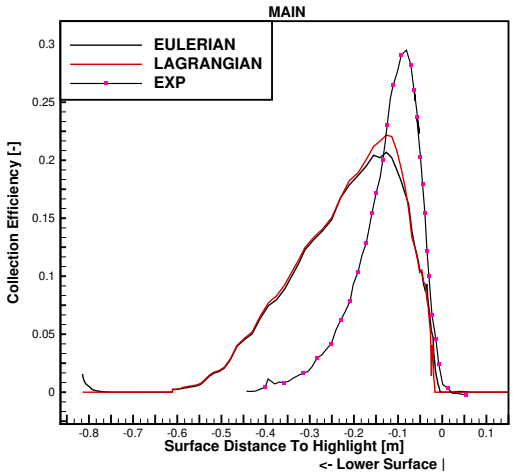
Figure 3.12 Aggregated impingement results of all IPW1 participants for Case IPW1-122.

The SLD results are presented in Fig. 3.13d–3.13f. On the slat, the predicted collection efficiency agrees well with the experimental measurements in terms of both the peak value and

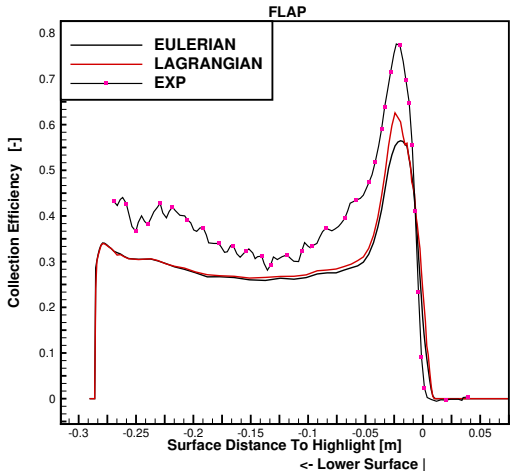
the impingement limits. The formulation of Trontin et al. underpredicts the peak collection efficiency but provides better agreement with the measurements over the remainder of the impingement region. On the main element, the collection efficiency is significantly underestimated, with only a minor influence from secondary-droplet re-impingement. Better agreement is obtained on the flap, where the impingement limits are well predicted, although the peak collection efficiency remains underestimated. In this case, including secondary-droplet re-impingement using the methodology presented in Section 3.3.3 noticeably increases the peak collection efficiency and improves the agreement with the experimental measurements. Similar discrepancies between numerical predictions and the experimental measurements have also been reported for this case in the literature [61, 143], particularly on the downstream elements.



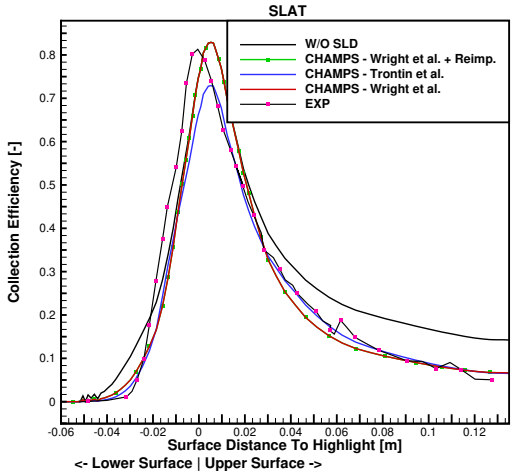
(a) Impingement results on the Slat.



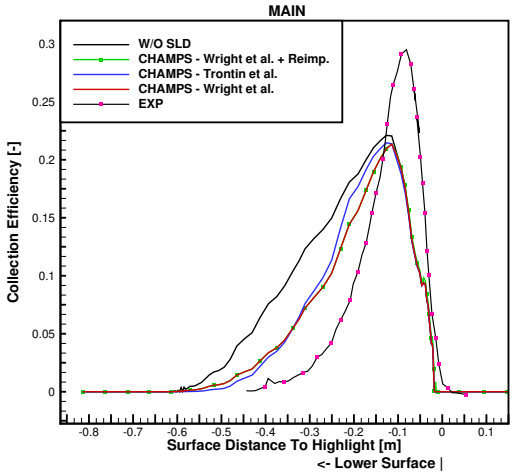
(b) Impingement results on the Main.



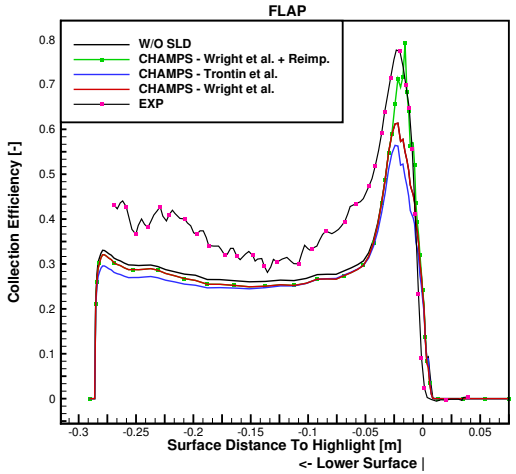
(c) Impingement results on the Flap.



(d) SLD results on the Slat.



(e) SLD results on the Main.



(f) SLD results on the Flap.

Figure 3.13 Collection-efficiency results for Case IPW1-122. The upper row compares Eulerian and Lagrangian impingement predictions; the lower row assesses the SLD models and secondary-droplet re-impingement.

3.4.4 Case HLPW5 - 2.1: CRM Wing-Body

The final case considers the Common Research Model (CRM) wing–body configuration to assess the Lagrangian methodology on a larger and more complex 3D geometry. The collection efficiency is extracted along five spanwise wing cuts and one fuselage cut, whose locations are shown in Fig. 3.14. The corresponding results for Case HLPW5 - 2.1 are presented in Fig. 3.15, where the Lagrangian predictions are compared with the Eulerian formulation.

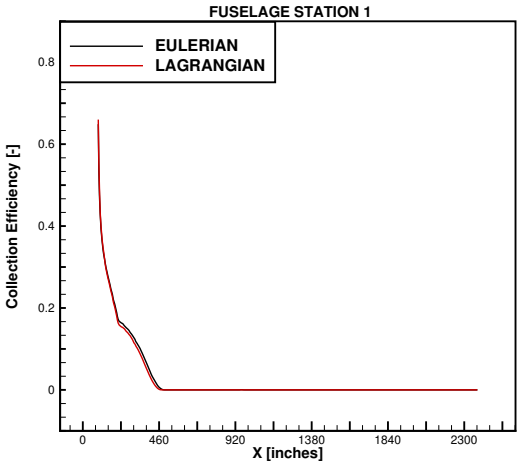
For the fuselage, as shown in Fig. 3.15a, the peak collection efficiency is obtained near the nose and progressively decreases along the surface. The collection-efficiency distributions along the wing are presented in Figs. 3.15b–3.15f. The distributions are asymmetric between the pressure and suction sides because of the prescribed angle of attack of 6°. Higher values of β are also observed near the wing–fuselage junction, indicating increased droplet impingement in this region. The distribution evolves along the span because of the swept-wing geometry and the corresponding variation in the local flow field. In particular, while impingement is predominantly observed on the pressure side at the inboard sections, the relative distribution between the pressure and suction sides changes farther outboard.

Overall, the Lagrangian and Eulerian formulations provide consistent collection-efficiency distributions across the fuselage and the five wing cuts. Good agreement is obtained in both the magnitude and spatial distribution of the collection efficiency, demonstrating the consistency of the two droplet formulations when applied to a large 3D configuration.

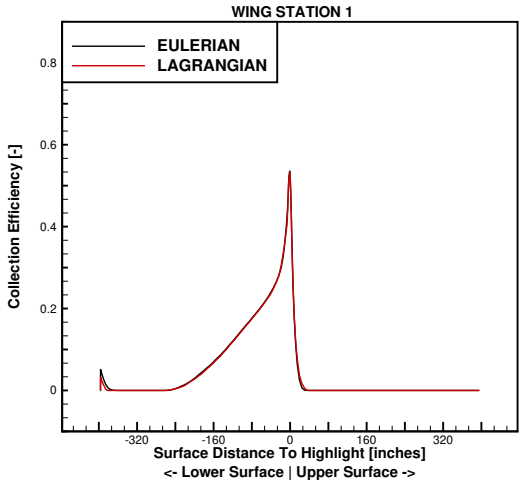


(a) Collection-efficiency cuts along the wing span. (b) Collection-efficiency cut on the fuselage.

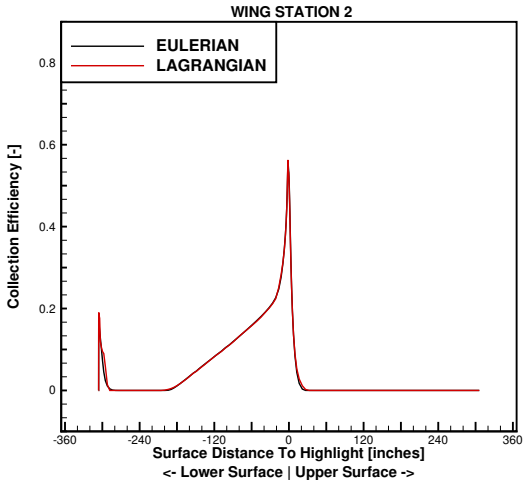
Figure 3.14 Locations of the collection-efficiency cuts on the CRM wing and fuselage.



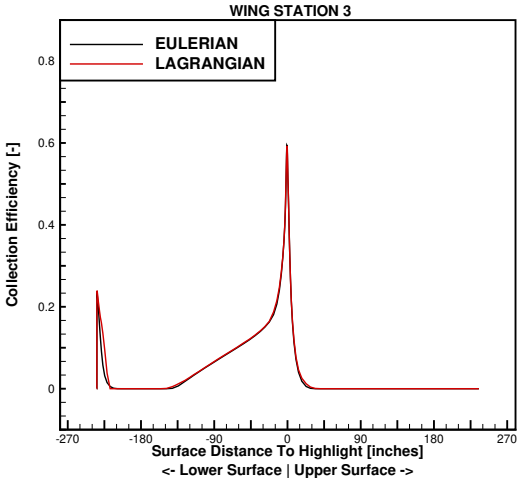
(a) Impingement results on the Fuselage.



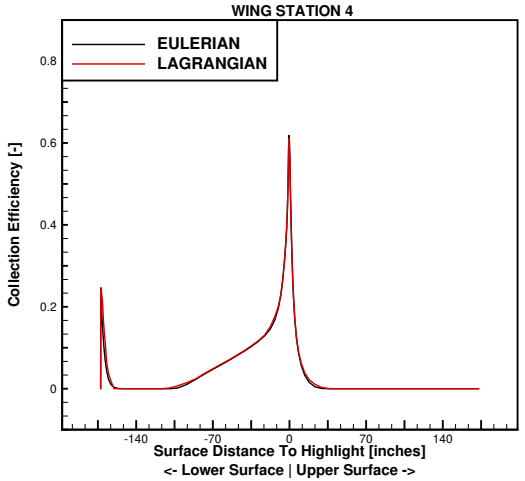
(b) Impingement results on the Wing-1.



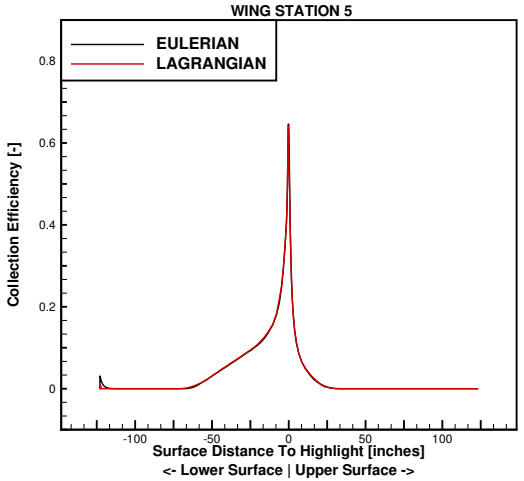
(c) Impingement results on the Wing-2.



(d) Impingement results on the Wing-3.



(e) Impingement results on the Wing-4.



(f) Impingement results on the Wing-5.

Figure 3.15 Eulerian and Lagrangian collection-efficiency results for Case HLPW5-2.1 at the fuselage and five spanwise wing cuts.

3.5 Conclusion

This chapter presents the development of a Lagrangian particle-tracking methodology for aircraft ice accretion simulations within the CHAMPS framework. The methodology combines face-to-face trajectory integration with a ray-tracing-enhanced intersection technique for robust and efficient tracking on unstructured meshes. An adaptive seeding strategy is introduced to progressively refine the droplet cloud in regions associated with surface impingement while reusing previously computed trajectories. The resulting workload is distributed according to estimates of the computational cost of individual trajectories to improve parallel execution on distributed-memory systems. For the evaluation of the collection efficiency, auxiliary trajectories are introduced locally around each seeding droplet to construct a local streamtube, while a neighboring-face interpolation treatment improves the coverage of the resulting surface distribution without requiring additional trajectories. SLD effects are also incorporated through mass-loss models and the tracking of secondary droplets generated by splashing.

The methodology is assessed using configurations from the AIAA Ice Prediction Workshops and the Fifth AIAA High-Lift Prediction Workshop. The parallel scalability study demonstrates a parallel efficiency of approximately 94% on 16 nodes relative to the two-node baseline. For the IPW1 configurations, the Lagrangian and Eulerian formulations provide consistent collection-efficiency distributions, while the inclusion of SLD effects improves agreement with the available experimental measurements. Finally, application to the CRM wing–body configuration demonstrates consistent Eulerian and Lagrangian predictions across the fuselage and multiple spanwise wing sections. Overall, the results demonstrate the capability of the developed methodology to provide robust and computationally efficient Lagrangian droplet-impingement predictions for complex 3D configurations.